

République du Sénégal



MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET DE LA RECHERCHE

DIRECTION GÉNÉRALE DE L'ENSEIGNEMENT SUPÉRIEUR



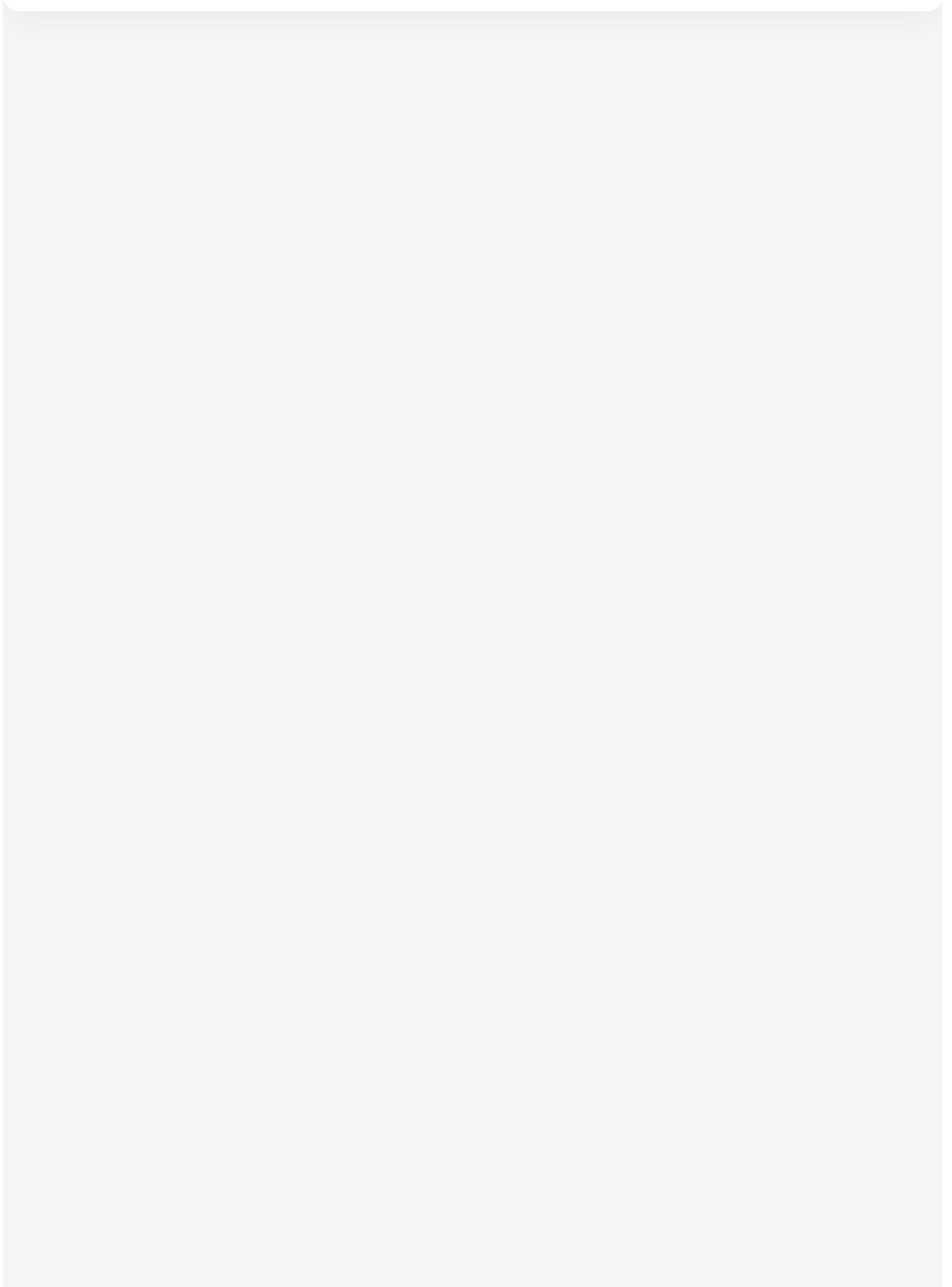
Rapport Kubernetes — Installation et administration d'un cluster

Présenté par : **Ibrahima Khaliloullah Thiam**

Master 2 Génie Logiciel UDB

Matricule 003

À **M. Massamba LO**



Sommaire

1. [Objectifs et contexte](#)
2. [Prérequis](#)
3. [Concepts et architecture](#)
4. [Installation et configuration](#)
5. [Réalisation pratique](#)
6. [Commandes et explications](#)
7. [Vérifications et tests](#)
8. [Dépannage](#)
9. [Sécurité et bonnes pratiques](#)
10. [Checklist](#)
11. [Conclusion](#)



1. Objectifs et contexte

Ce travail pratique a pour objectif d'étudier Kubernetes, plateforme open source d'orchestration de conteneurs, puis de mettre en œuvre un cluster Linux avec **kubeadm**, **containerd**, **kubelet** et **kubectl**.

La réalisation repose sur un nœud *control plane* et au moins un nœud *worker*. Le control plane expose l'API Kubernetes et coordonne le cluster ; les workers exécutent les charges de travail conteneurisées.

- Comprendre le rôle de Kubernetes dans une architecture conteneurisée.
- Identifier les principaux composants du control plane et des workers.
- Installer un runtime compatible CRI, ici **containerd**.
- Installer **kubeadm**, **kubelet** et **kubectl**.
- Initialiser le control plane et déployer un plugin CNI.
- Joindre un worker au cluster et vérifier son état.
- Déployer une application simple afin de valider le fonctionnement.

Cadre technique : Kubernetes recommande **kubeadm** pour construire un cluster administré soi-même. Les procédures modernes utilisent **pkgs.k8s.io** ; l'ancien dépôt **apt.kubernetes.io** est déprécié.

2. Prérequis

2.1 Infrastructure minimale

Nœud	CPU	RAM	Rôle	Exemple IP
k8s-master	2 vCPU ou plus	2 Go ou plus	Control plane	10.0.1.10
k8s-worker1	2 vCPU recommandé	2 Go ou plus	Worker	10.0.1.20

La documentation officielle demande notamment au moins 2 GiB de mémoire par machine, au moins 2 CPU pour le control plane, une connectivité réseau complète et des identifiants système uniques pour chaque nœud.

2.2 Logiciels et accès

- Ubuntu/Debian ou distribution Linux compatible.
- Accès administrateur avec `sudo`.
- Connectivité réseau entre les nœuds et accès aux dépôts.
- Nom d'hôte distinct pour chaque machine.
- Synchronisation de l'heure.

2.3 Vérifications initiales

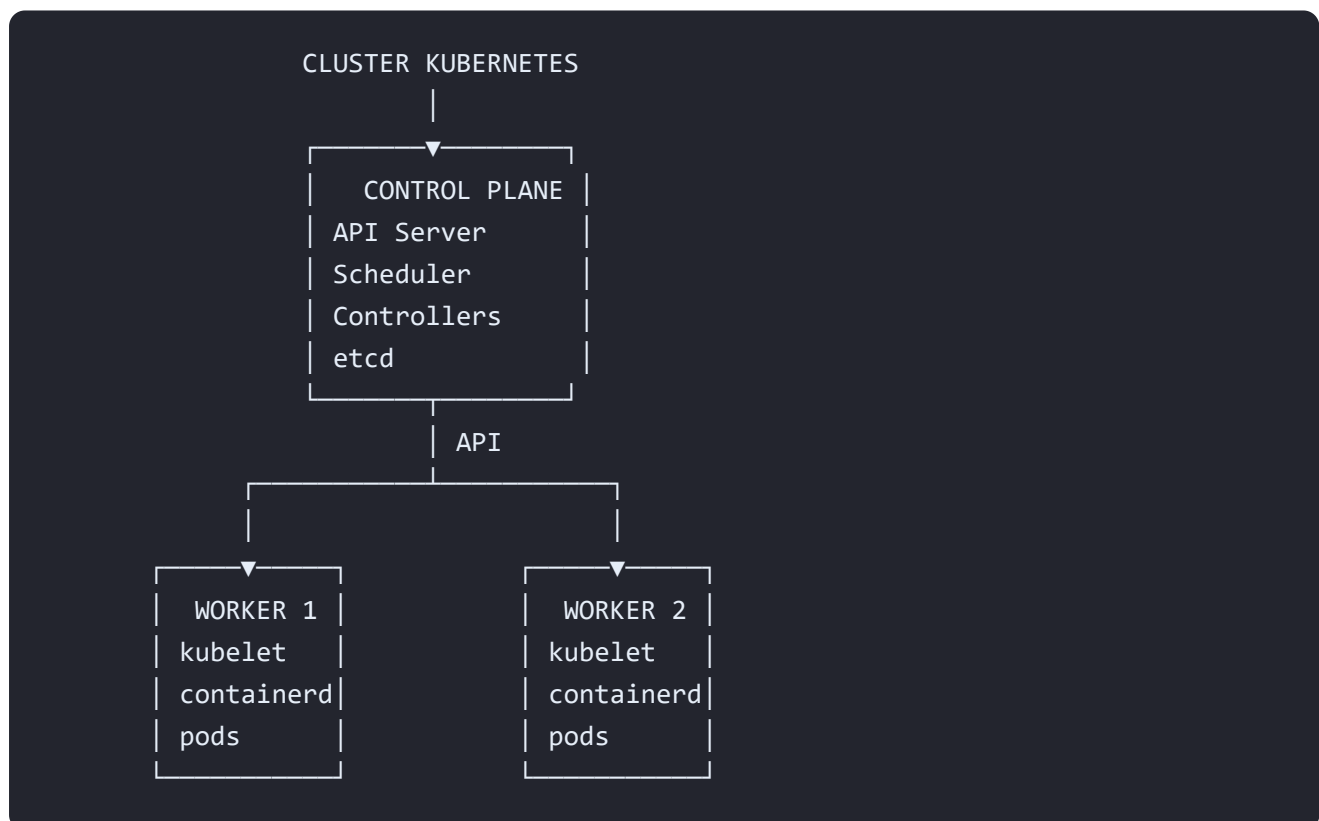
```
hostnamectl
uname -r
ip addr
ip route
free -h
nproc
lsb_release -a
```

Important : choisir une version mineure Kubernetes et conserver une cohérence de version entre `kubeadm`, `kubelet` et le control plane.

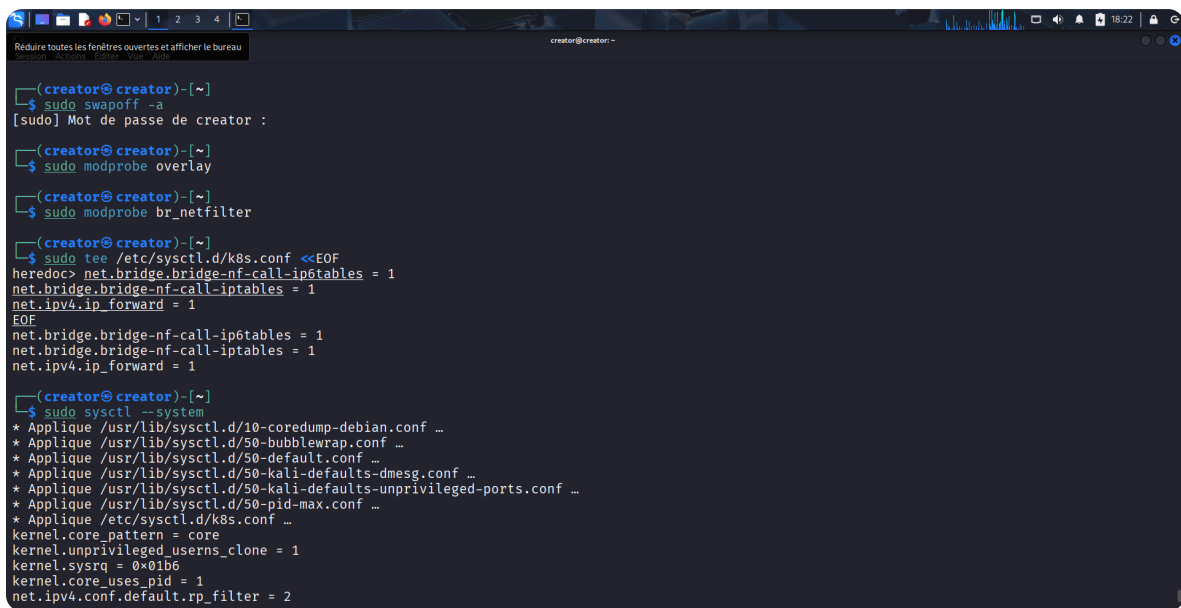
3. Concepts et architecture

Kubernetes transforme un ensemble de machines en cluster capable d'exécuter et de maintenir des applications conteneurisées selon un état désiré.

Composant	Rôle
API Server	Point d'entrée de l'API Kubernetes et interface centrale du control plane.
etcd	Stockage de l'état du cluster.
Scheduler	Sélectionne le nœud des nouveaux pods.
Controller Manager	Rapproche l'état réel de l'état désiré.
kubelet	Agent présent sur chaque nœud.
kube-proxy	Participe à la connectivité des Services.
containerd	Runtime de conteneurs utilisé via CRI.
CNI	Fournit le réseau des pods.
Pod	Plus petite unité déployable.
Service	Point d'accès stable vers des pods.



Architecture réseau : le CNI fournit la connectivité des pods. Le CIDR choisi doit être cohérent avec le plugin réseau et ne pas entrer en conflit avec les réseaux existants.



```
(creator@creator)-[~]
$ sudo swapoff -a
[sudo] Mot de passe de creator :

(creator@creator)-[~]
$ sudo modprobe overlay

(creator@creator)-[~]
$ sudo modprobe br_netfilter

(creator@creator)-[~]
$ sudo tee /etc/sysctl.d/k8s.conf <<EOF
heredoc> net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
EOF
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1

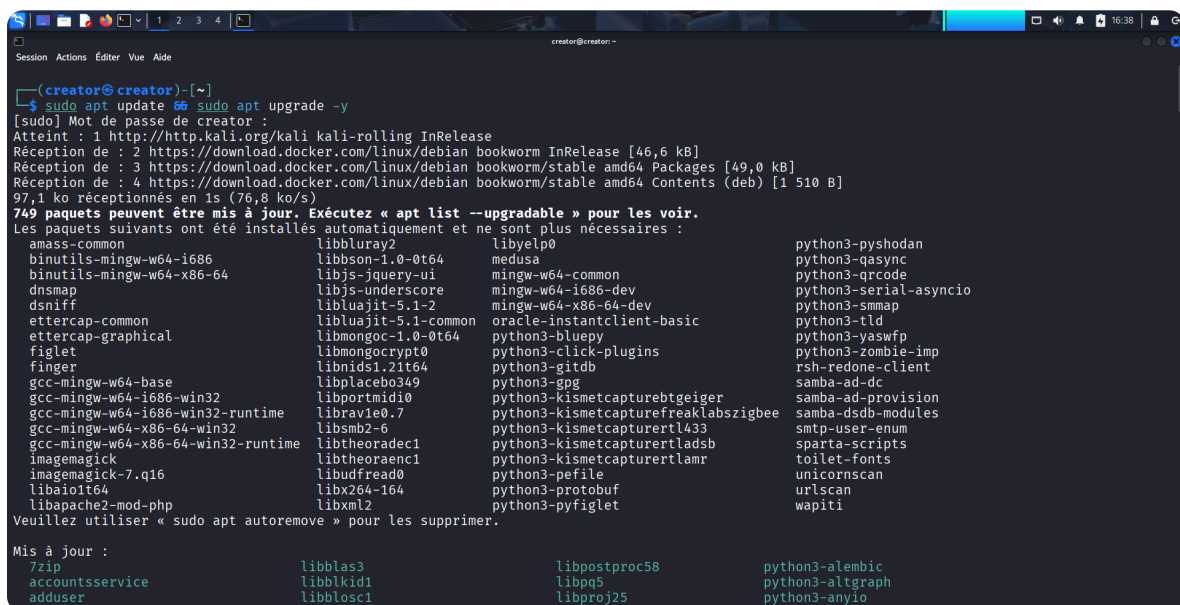
(creator@creator)-[~]
$ sudo sysctl --system
* Applique /usr/lib/sysctl.d/10-coredump-debian.conf ...
* Applique /usr/lib/sysctl.d/50-bubblewrap.conf ...
* Applique /usr/lib/sysctl.d/50-default.conf ...
* Applique /usr/lib/sysctl.d/50-kali-defaults-dmesg.conf ...
* Applique /usr/lib/sysctl.d/50-kali-defaults-unprivileged-ports.conf ...
* Applique /usr/lib/sysctl.d/50-pid-max.conf ...
* Applique /etc/sysctl.d/k8s.conf ...
kernel.core_pattern = core
kernel.unprivileged_userns_clone = 1
kernel.sysrq = 0x01b6
kernel.core_uses_pid = 1
net.ipv4.conf.default.rp_filter = 2
```

4. Installation et configuration

4.1 Préparer chaque nœud

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl gpg
sudo swapoff -a
sudo sed -i '/[[:space:]]swap[[:space:]]/ s/^/#/' /etc/fstab
cat <
```

Explications : le swap est désactivé, les modules réseau sont chargés et le routage IPv4 est activé pour préparer le nœud à Kubernetes.



```
(creator@creator)-[~]
$ sudo apt update && sudo apt upgrade -y
[sudo] Mot de passe de creator :
Atteint : 1 http://http.kali.org/kali kali-rolling InRelease
Réception de : 2 https://download.docker.com/linux/debian bookworm InRelease [46,6 kB]
Réception de : 3 https://download.docker.com/linux/debian bookworm/stable amd64 Packages [49,0 kB]
Réception de : 4 https://download.docker.com/linux/debian bookworm/stable amd64 Contents (deb) [1 510 B]
97,1 ko réceptionnés en 1s (76,8 ko/s)
749 paquets peuvent être mis à jour. Exécutez « apt list --upgradable » pour les voir.
Les paquets suivants ont été installés automatiquement et ne sont plus nécessaires :
amass-common libbluray2 libyelp0 python3-pyshodan
binutils-mingw-w64-i686 libbson-1.0-0t64 medusa python3-qasync
binutils-mingw-w64-x86_64 libjs-jquery-ui mingw-w64-common python3-qrcode
dnsmasq libjs-underscore mingw-w64-i686-dev python3-serial-asyncio
dsniff liblua5.1-2 mingw-w64-x86_64-dev python3-smmmap
ettercap-common liblua5.1-common oracle-instantclient-basic python3-tld
ettercap-graphical libmongoc-1.0-0t64 python3-bluepy python3-yaswfp
figlet libmongocrypt0 python3-click-plugins python3-zombie-imp
finger libnids1.21t64 python3-gitdb rsh-redone-client
gcc-mingw-w64-base libplacebo349 python3-gpg samba-ad-dc
gcc-mingw-w64-i686-win32 libportmidi0 python3-kismetcapturebtgeiger samba-ad-provision
gcc-mingw-w64-i686-win32-runtime librav1e0.7 python3-kismetcapturefreaklabszigbee samba-dsdb-modules
gcc-mingw-w64-x86_64-win32 libsmb2-6 python3-kismetcapturertl433 smtp-user-enum
gcc-mingw-w64-x86_64-win32-runtime libtheoradec1 python3-kismetcapturertladsb sparta-scripts
imagemagick libtheoraenc1 python3-kismetcapturertlamr toilet-fonts
imagemagick-7.q16 libudfread0 python3-pefile unicorncan
libaio1t64 libx264-164 python3-protobuf urlscan
libapache2-mod-php libxml2 python3-pyfiglet wapiti
Veuillez utiliser « sudo apt autoremove » pour les supprimer.

Mis à jour :
7zip libblas3 libpostproc58 python3-alembic
accountsservice libblkid1 libpq5 python3-altgraph
adduser libblosc1 libproj25 python3-anyio
```

4.2 Installer et configurer containerd

```
sudo apt-get update
sudo apt-get install -y containerd
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml >/dev/null
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
sudo systemctl restart containerd
sudo systemctl enable containerd
sudo systemctl status containerd --no-pager
```

Runtime : Kubernetes utilise le CRI pour communiquer avec un runtime. containerd est compatible avec cette architecture ; Docker Engine seul ne fournit pas directement le CRI attendu.

```
creator@creator:~$ sudo apt install -y containerd
Les paquets suivants ont été installés automatiquement et ne sont plus nécessaires :
amass-common libbluray2 libbyp0 python3-qasync
binutils-mingw-w64-i686 libbson-1.0-0t64 medusa python3-qrcode
binutils-mingw-w64-x86-64 libjs-jquery-ui mingw-w64-common python3-serial-asyncio
dnsmasq libjs-underscore mingw-w64-i686-dev python3-smbmap
docker-ce-rootless-extras liblua5.1-2 mingw-w64-x86-64-dev python3-tld
dsniff liblua5.1-common oracle-instantclient-basic python3-yaswfp
ettercap-common libmongoc-1.0-0t64 pigz python3-bluepy python3-zombie-imp
ettercap-graphical libmongocrypt0 python3-click-plugins rsh-redone-client
figlet libnids1.21t64 python3-gitdb samba-ad-dc
finger libplacebo349 python3-gpg samba-ad-provision
gcc-mingw-w64-base libportmidi0 python3-kismetcapturebtgeiger samba-dsdb-modules
gcc-mingw-w64-i686-win32 librav1e0.7 python3-kismetcapturefreaklabszigbee slirp4netns
gcc-mingw-w64-i686-win32-runtime libslirp0 python3-kismetcapturefreaklabszigbee smtp-user-enum
gcc-mingw-w64-x86-64-win32 libsnb2-6 python3-kismetcapturefreaklabszigbee sparta-scripts
gcc-mingw-w64-x86-64-win32-runtime libtheoradec1 python3-kismetcapturefreaklabszigbee toilet-fonts
imagemagick libtheoraenc1 python3-kismetcapturefreaklabszigbee unicornscan
imagemagick-7.q16 libudfread0 python3-pefile uriscan
libaio1t64 libx264-164 python3-pyfiglet wapiti
libapache2-mod-php libxml2 python3-pyshodan

Veuillez utiliser « sudo apt autoremove » pour les supprimer.

Installation de :
containerd

Installation de dépendances :
criu libcompell python3-pycriu runc

Paquets suggérés :
containernetworking-plugins

SUPPRESSION :
```

4.3 Installer kubeadm, kubelet et kubectl

L'ancien dépôt `apt.kubernetes.io` du rapport précédent est remplacé par `pkgs.k8s.io`. L'exemple utilise Kubernetes **1.37** ; pour une autre version mineure, remplacer le numéro dans l'URL du dépôt.

```
sudo mkdir -p -m 755 /etc/apt/keyrings
curl -fsSL https://pkgs.k8s.io/core:stable:/v1.37/deb/Release.key \
| sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:stable:/v1.37/deb/
| sudo tee /etc/apt/sources.list.d/kubernetes.list
sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
sudo systemctl enable --now kubelet
kubeadm version
kubelet --version
kubectl version --client
```

Rôle des outils : `kubeadm` initialise/rejoint les clusters, `kubelet` maintient les pods et `kubectl` dialogue avec l'API Kubernetes.

```
Session [x] Émulateur de terminal [x] Utiliser la ligne de commande [x] creator@creator:--

(creator@creator)-[~]
$ sudo apt install -y apt-transport-https ca-certificates curl
ca-certificates est déjà la version la plus récente (20250419).
curl est déjà la version la plus récente (8.15.0-1).
Les paquets suivants ont été installés automatiquement et ne sont plus nécessaires :
amass-common libbluray2 libyelp0 python3-qasync
binutils-mingw-w64-i686 libbson-1.0-0t64 medusa python3-qrcode
binutils-mingw-w64-x86-64 libjs-jquery-ui mingw-w64-common python3-serial-asyncio
dnsmap libjs-underscore mingw-w64-i686-dev python3-smmmap
docker-ce-rootless-extras liblua5.1-2 mingw-w64-x86-64-dev python3-tld
dsniff liblua5.1-common oracle-instantclient-basic python3-yaswfp
ettercap-common libmongoc-1.0-0t64 pigz python3-bluepy python3-zombie-imp
ettercap-graphical libnids1.21t64 python3-click-plugins rsh-redone-client
figlet libplacebo349 python3-gitdb samba-ad-dc
finger libportmidi0 python3-gpg samba-ad-provision
gcc-mingw-w64-base libray1e0.7 python3-kismetcapturebtgeiger samba-dsdb-modules
gcc-mingw-w64-i686-win32 libslirp0 python3-kismetcapturefreaklabszigbee slirp4netns
gcc-mingw-w64-i686-win32-runtime libsm2-6 python3-kismetcapturertl433 smtp-user-enum
gcc-mingw-w64-x86-64-win32 libtheoradec1 python3-kismetcapturertladsb sparta-scripts
gcc-mingw-w64-x86-64-win32-runtime libtheoraenc1 python3-kismetcapturertlamr toilet-fonts
imagemagick libudfread0 python3-pefile unicorncan
imagemagick-7.q16 libx264-164 python3-pyfiglet urlscan
libaio1t64 libxml2 python3-pyshodan wapiti
libapache2-mod-php libxml2

Veuillez utiliser « sudo apt autoremove » pour les supprimer.

Installation de :
  apt-transport-https

Sommaire :
  Mise à niveau de : 0. Installation de : 1Supprimé : 0. Non mis à jour : 4
  Taille du téléchargement : 40,8 kB
  Espace nécessaire : 52,2 kB / 1 276 MB disponible
```

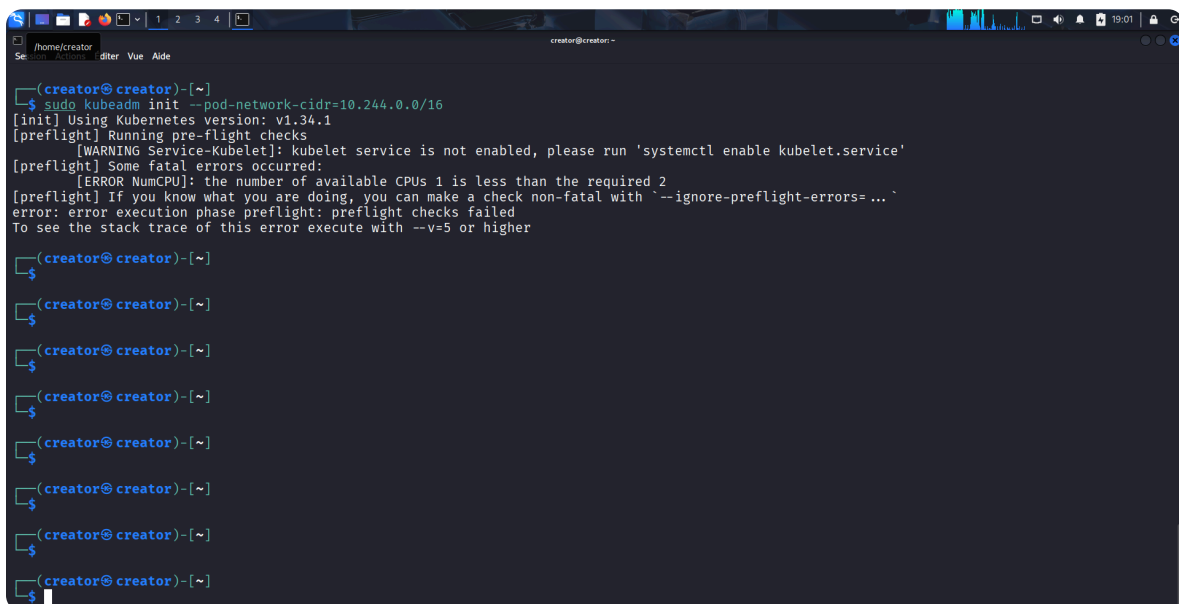
5. Réalisation pratique

5.1 Initialisation du control plane

Sur k8s-master, initialiser le control plane. Le CIDR ci-dessous correspond au scénario Flannel du TP.

```
sudo kubeadm init --pod-network-cidr=10.244.0.0/16
```

Résultat : kubeadm effectue les contrôles préalables, génère les certificats, déploie les composants du control plane et fournit les instructions de configuration de kubectl ainsi que la commande de jonction des workers.



```
(creator@creator)-[~]
$ sudo kubeadm init --pod-network-cidr=10.244.0.0/16
[init] Using Kubernetes version: v1.34.1
[preflight] Running pre-flight checks
[WARNING Service-Kubelet]: kubelet service is not enabled, please run 'systemctl enable kubelet.service'
[preflight] Some fatal errors occurred:
[ERROR NumCPU]: the number of available CPUs 1 is less than the required 2
[preflight] If you know what you are doing, you can make a check non-fatal with '--ignore-preflight-errors=...'
error: error execution phase preflight: preflight checks failed
To see the stack trace of this error execute with --v=5 or higher

(creator@creator)-[~]
$
(creator@creator)-[~]
$
(creator@creator)-[~]
$
(creator@creator)-[~]
$
(creator@creator)-[~]
$
(creator@creator)-[~]
$
(creator@creator)-[~]
$
```

5.2 Configurer kubectl

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown "$(id -u):$(id -g)" $HOME/.kube/config
kubectl get nodes
```

5.3 Installer le réseau CNI

Un cluster kubeadm a besoin d'un plugin réseau. L'exemple conserve Flannel afin de rester cohérent avec le TP initial.

```
kubectl apply -f https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml
kubectl get pods -n kube-flannel
kubectl get nodes
```

Attention : ne pas déployer plusieurs CNI simultanément dans ce scénario. Le CIDR et la procédure doivent correspondre au plugin retenu.

5.4 Joindre le worker

Sur le worker, exécuter la commande générée par `kubeadm init` :

```
sudo kubeadm join 10.0.1.10:6443 \  
  --token <TOKEN> \  
  --discovery-token-ca-cert-hash sha256:<HASH>
```

Paramètres : 10.0.1.10:6443 désigne l'API Server ; le token sert à l'amorçage et le hash protège la découverte de l'autorité de certification.

5.5 Déployer une application de test

```
kubectl create deployment nginx --image=nginx:stable  
kubectl expose deployment nginx --port=80 --type=NodePort  
kubectl get deployments  
kubectl get pods -o wide  
kubectl get services
```

Cette étape valide la chaîne API Kubernetes → scheduler → kubelet → containerd → pod → Service.

6. Commandes et explications

6.1 Diagnostic

```
kubectl get nodes -o wide
kubectl get pods -A
kubectl get svc -A
kubectl get events -A --sort-by=.lastTimestamp
kubectl cluster-info
```

6.2 Inspection

```
kubectl describe node k8s-worker1
kubectl describe pod <NOM_DU_POD>
kubectl logs <NOM_DU_POD>
kubectl logs <NOM_DU_POD> --previous
```

6.3 Déploiements

```
kubectl scale deployment nginx --replicas=3
kubectl rollout status deployment/nginx
kubectl rollout history deployment/nginx
kubectl rollout undo deployment/nginx
kubectl delete service nginx
kubectl delete deployment nginx
```

6.4 kubeadm

```
kubeadm token create --print-join-command
sudo kubeadm reset
kubeadm version
kubectl version --client
```

Prudence : kubeadm reset est une opération destructive à réserver à une procédure de réinitialisation contrôlée.

7. Vérifications et tests

7.1 État des nœuds

```
kubectl get nodes -o wide
```

Les nœuds attendus doivent apparaître avec le statut **Ready**.

7.2 Pods système

```
kubectl get pods -n kube-system  
kubectl get pods -n kube-flannel
```

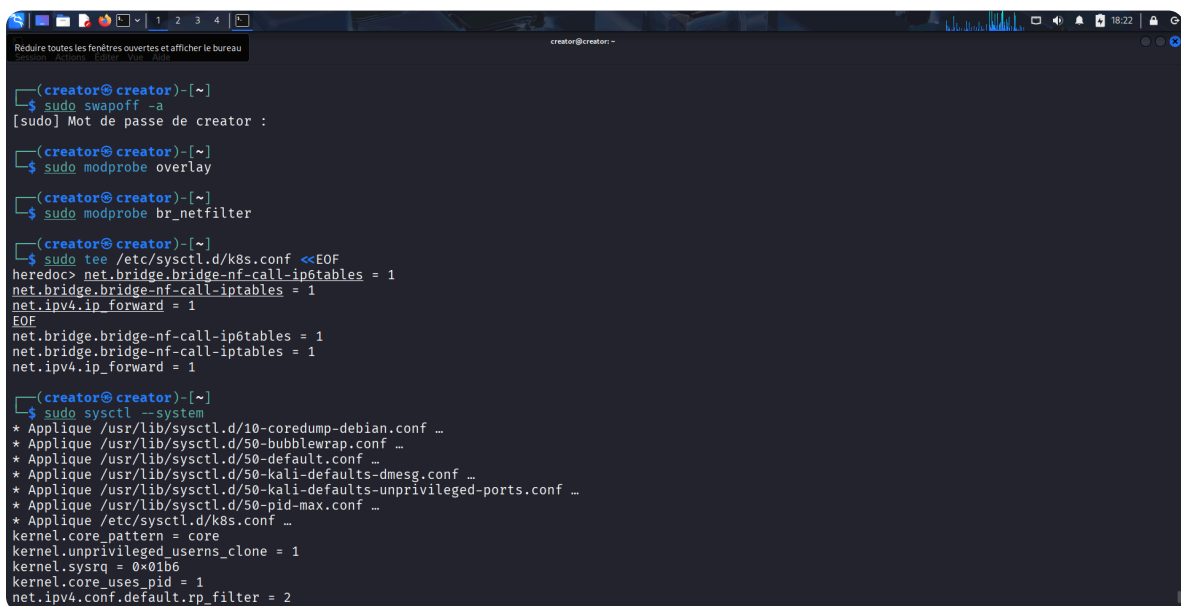
7.3 Tester Nginx

```
kubectl get deployment nginx  
kubectl get pods -l app=nginx -o wide  
kubectl get service nginx  
curl http://<IP_NOEUD>:<NODE_PORT>
```

Le test est réussi si le Service fournit la réponse HTTP de Nginx et que les pods restent disponibles.

7.4 Vérifier les services système

```
sudo systemctl status containerd --no-pager  
sudo systemctl status kubelet --no-pager  
sudo journalctl -u kubelet -n 100 --no-pager
```



```
(creator@creator)-[~]  
$ sudo swapoff -a  
[sudo] Mot de passe de creator :  
  
(creator@creator)-[~]  
$ sudo modprobe overlay  
  
(creator@creator)-[~]  
$ sudo modprobe br_netfilter  
  
(creator@creator)-[~]  
$ sudo tee /etc/sysctl.d/k8s.conf <<EOF  
heredoc> net.bridge.bridge-nf-call-ip6tables = 1  
net.bridge.bridge-nf-call-iptables = 1  
net.ipv4.ip_forward = 1  
EOF  
net.bridge.bridge-nf-call-ip6tables = 1  
net.bridge.bridge-nf-call-iptables = 1  
net.ipv4.ip_forward = 1  
  
(creator@creator)-[~]  
$ sudo sysctl --system  
* Applique /usr/lib/sysctl.d/10-coredump-debian.conf ...  
* Applique /usr/lib/sysctl.d/50-bubblewrap.conf ...  
* Applique /usr/lib/sysctl.d/50-default.conf ...  
* Applique /usr/lib/sysctl.d/50-kali-defaults-dmesg.conf ...  
* Applique /usr/lib/sysctl.d/50-kali-defaults-unprivileged-ports.conf ...  
* Applique /usr/lib/sysctl.d/50-pid-max.conf ...  
* Applique /etc/sysctl.d/k8s.conf ...  
kernel.core_pattern = core  
kernel.unprivileged_usersns_clone = 1  
kernel.sysrq = 0x01b6  
kernel.core_uses_pid = 1  
net.ipv4.conf.default.rp_filter = 2
```

8. Dépannage

Symptôme	Cause possible	Diagnostic	Correction
Nœud NotReady	CNI absent/défaillant	<code>kubectl get pods -A</code>	Vérifier le CNI et ses logs.
kubelet en échec	Runtime/configuration	<code>journalctl -u kubelet</code>	Vérifier containerd, cgroups et réseau.
Worker impossible à joindre	Port 6443 bloqué	<code>nc -vz <MASTER_IP> 6443</code>	Autoriser le trafic nécessaire.
Pod Pending	Ressources/contraintes	<code>kubectl describe pod</code>	Lire les événements et vérifier les ressources.
Image inaccessible	DNS/réseau/registre	<code>kubectl describe pod</code>	Tester DNS et accès au registre.
Service inaccessible	NodePort/firewall/CNI	<code>kubectl get svc</code>	Vérifier port, firewall et endpoints.

Diagnostic systématique

```
kubectl get nodes
kubectl get pods -A
kubectl get events -A --sort-by=.lastTimestamp
sudo systemctl status kubelet --no-pager
sudo systemctl status containerd --no-pager
sudo journalctl -u kubelet -n 200 --no-pager
sudo journalctl -u containerd -n 200 --no-pager
```

Méthode : diagnostiquer d'abord l'état global, puis la ressource en erreur, puis les événements et enfin les logs du composant responsable.

9. Sécurité et bonnes pratiques

9.1 Réseau et pare-feu

Les ports doivent être ouverts uniquement entre les machines et sources nécessaires. Les ports standards comprennent notamment 6443/TCP pour l'API Server, 2379-2380/TCP pour etcd, 10250/TCP pour kubelet et 30000-32767 pour les NodePorts.

```
sudo ss -ltnp  
sudo ufw status verbose
```

9.2 Identité et accès

- Protéger `/etc/kubernetes/admin.conf`.
- Éviter l'utilisation permanente d'un kubeconfig administrateur.
- Utiliser RBAC avec le moindre privilège.
- Créer des ServiceAccounts dédiés.

9.3 Images et workloads

- Utiliser des registres fiables.
- Épingler les versions des images plutôt que `:latest`.
- Scanner les images avant déploiement.
- Éviter les conteneurs privilégiés sans nécessité.
- Ne pas stocker de secrets dans les images.

9.4 Maintenance

- Planifier les mises à niveau de Kubernetes et du runtime.
- Surveiller CPU, mémoire, disque et réseau.
- Centraliser les logs et alertes.
- Sauvegarder les données critiques, notamment etcd selon l'architecture.

Principe fondamental : la sécurité Kubernetes couvre l'identité, le réseau, les images, les workloads, les secrets, les logs et les procédures d'administration.

10. Checklist

Validation du TP Kubernetes

- ☐ Noms d'hôtes et adresses IP vérifiés.
- ☐ Ressources CPU/RAM suffisantes.
- ☐ Swap désactivé.
- ☐ Modules `overlay` et `br_netfilter` chargés.
- ☐ Paramètres `sysctl` appliqués.
- ☐ `containerd` actif avec `systemd cgroups`.
- ☐ `kubeadm`, `kubelet` et `kubectrl` installés depuis `pkgs.k8s.io`.
- ☐ Control plane initialisé.
- ☐ `kubeconfig` configuré.
- ☐ CNI installé.
- ☐ Worker joint au cluster.
- ☐ Tous les nœuds attendus sont `Ready`.
- ☐ Pods système opérationnels.
- ☐ Déploiement `Nginx` créé.
- ☐ Service `NodePort` testé.
- ☐ Logs vérifiés en cas d'erreur.
- ☐ Règles `firewall` contrôlées.

11. Conclusion

Ce travail pratique a permis de passer d'une présentation générale de Kubernetes à la réalisation d'un cluster fonctionnel reposant sur un control plane et un nœud worker. La démarche a couvert la préparation du système Linux, l'installation de containerd, l'installation de kubeadm/kubelet/kubectl, l'initialisation du control plane, le réseau CNI et l'intégration d'un worker.

Les vérifications réalisées avec kubectl, les journaux de kubelet et de containerd, ainsi que le déploiement d'une application Nginx permettent de valider les principales briques du cluster.

Au-delà de l'installation, Kubernetes impose une démarche d'exploitation structurée : RBAC, segmentation réseau, gestion des secrets, supervision, sauvegarde et mises à jour contrôlées. Le cluster constitue ainsi une base pédagogique pour approfondir Ingress, stockage persistant, Helm, Prometheus/Grafana et les architectures hautement disponibles.

Références techniques : documentation officielle Kubernetes relative à l'installation de kubeadm, la création d'un cluster, l'ajout de workers et les ports réseau.

